MECHATRONIC SYSTEM INTEGRATION

(MCTA 3203)

SEMESTER 1, 2026/2027

EXPERIMENT 2

SERIAL COMUNICATION BETWEEN ARDUINO AND PYTHON

(MCTA 3203)

SECTION 1

GROUP 4

NO.	MATRIC NO.	NAME
1	2413381	HARITH NAUFAL BIN AHMAD SHUKRI
2	2418123	IMAN BIN SALMAN
3	2411349	LUQMAN HAKIMI BIN SUHAIZI
4	2418929	MUHAMAD HAZIQ BIN MOHD SHAHRIL

SUBMISSION DATE: 19 OCTOBBER 2026

ABSTRACT

This experiment demonstrates the integration of serial communication between Arduino and Python through two main tests: reading potentiometer values and controlling the angle of a servo motor. In the first experiment, the potentiometer value is read by the Arduino via analog input pin A0 and transmitted to a Python script using the pyserial library. The second experiment focuses on computer control, where an angle input (0 to 180 degrees) is entered in the Python terminal and sent to the Arduino to move a servo motor connected to pin 9. The primary objectives of this experiment are to transmit readings from the hardware (Arduino) to the software (Python) and to control the motor using input from the computer.

During execution, communication functions properly when the correct code is uploaded and the baud rate is synchronized at 9600. However, a highlighted precaution is that the Arduino Serial Plotter must not be opened simultaneously while the Python script is running, as it will interfere with Python's access to the serial port. As system improvements and extended tasks, the experiment is further developed by adding an LED component (with a 220 Ω resistor) that lights up automatically based on the potentiometer value limit or servo position, using the potentiometer for real-time servo adjustment, and integrating graphical visualization using the matplotlib library in Python to plot reading data in real-time.

TABLE OF CONTENTS

ABSTRACT....2
TABLE OF CONTENTS....3
INTRODUCTION & THEORY....4
MATERIALS AND EQUIPMENT....5
EXPERIMENTAL SETUP....6
METHODOLOGY & PROGRAMMING LOGIC....8
DATA COLLECTION & ANALYSIS....13
RESULTS & DISCUSSION....15
CONCLUSION....17
RECOMMENDATIONS....18
REFERENCES....19
APPENDICES....20
ACKNOWLEDGEMENT....21
STUDENT DECLARATION....22

INTRODUCTION & THEORY

Serial communication enables continuous data transfer between a microcontroller and a computer system. In this experiment, an Arduino board was used to interface with a potentiometer, where the center pin was connected to the analog input pin A0 to capture variable analog readings.

Rather than displaying outputs on external hardware components, the Arduino was programmed to read the analog values and transmit them to a computer via a USB serial connection configured at a 9600 baud rate. On the receiving end, a Python script utilizing the pyserial library was used to establish communication, continuously decode the incoming data stream, and print the real time potentiometer values directly to the terminal.

The primary objective of this experiment was to successfully transmit potentiometer readings from the Arduino to Python via USB serial, establishing a reliable hardware to software communication pipeline.

MATERIALS AND EQUIPMENT

The following components were used:

- Arduino board
- Potentiometer
- Servo motor
- LED
- 220 Ω resistors
- Jumper wires
- Breadboard
- Laptop with Arduino IDE and Pycharm

EXPERIMENTAL SETUP

Experiment 1

- Potentiometer is connected to the Arduino at pin A0 using it's center pin. The left pin of the potentiometer is connected to ground while the right pin is connected to 5v. (left pin → GND, output pin → pin A0, right → pin 5V)
- Hardware wiring diagram:

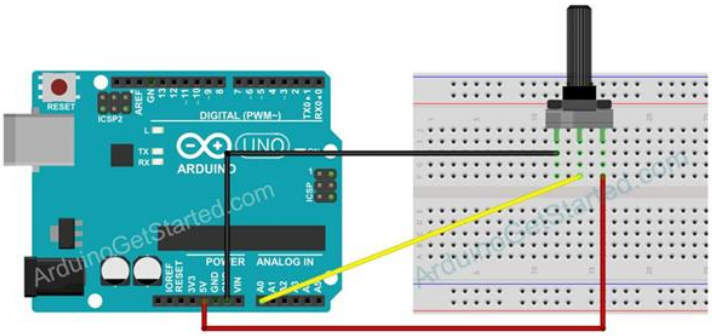


Figure 1.1

Task 1

- Potentiometer is connected to the Arduino at pin A0 using it's center pin. The left pin of the potentiometer is connected to ground while the right pin is connected to 5v. (left pin → GND, output pin → pin A0, right → pin 5V)
- Anode leg of LED is connected to a 220-ohm resistor to reduce the voltage from 5v to 3v. The cathode leg is connected to ground. (anode leg → 220-ohm resistor → pin 8, cathode leg → GND)
- The resistor is connected to the Arduino at pin 8.

Experiment 2

- The servo motor's pwm pin is connected to the Arduino at pin 9. The 5v and ground pin are connected to the 5v and ground pin on the Arduino. (Signal → pin 9, Vcc → 5V, GND → GND)
- Hardware wiring diagram:

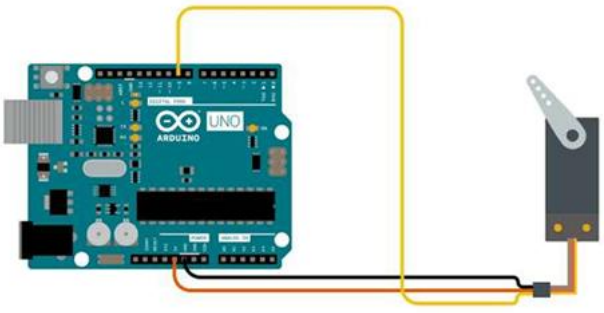


Figure 2.1

Task 2

- Potentiometer is connected to the Arduino at pin A0 using it's center pin. The left pin of the potentiometer is connected to ground while the right pin is connected to 5v. (left pin → GND, output pin → pin A0, right → pin 5V)
- The servo motor's pwm pin is connected to the Arduino at pin 9. The 5v and ground pin are connected to the 5v and ground pin on the Arduino. (Signal → pin 9, Vcc → 5V, GND → GND)
- Anode leg of LED is connected to a 220-ohm resistor to reduce the voltage from 5v to 3v. The cathode leg is connected to ground. (anode leg → 220-ohm resistor → pin 8, cathode leg → GND)
- The resistor is connected to the Arduino at pin 8.
- Hardware wiring diagram:

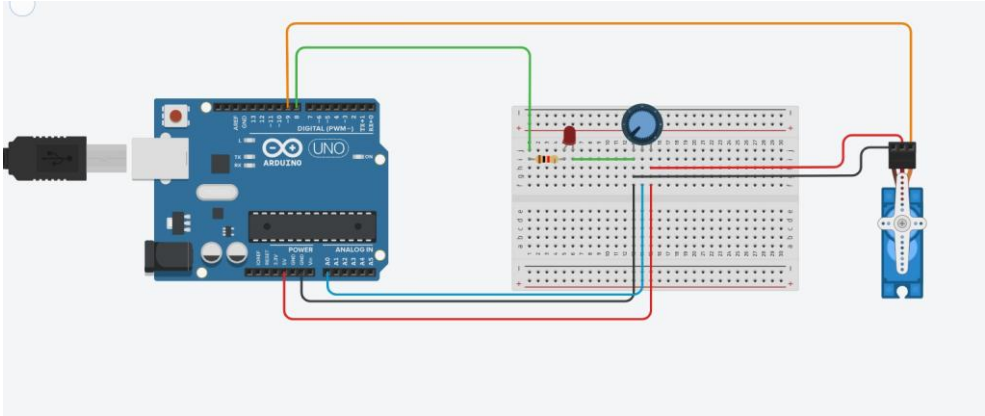


Figure 2.2

METHODOLOGY & PROGRAMMING LOGIC

Experiment 1

The experiment was carried out by:

1. Connecting the potentiometer to the Arduino according to the required connections.
2. Writing and uploading an Arduino program to read the value of the potentiometer and print it over the serial port.
3. Using Pycharm, a python script is written to read the value sent over the serial port and display the value.

Arduino C++ code:

- “int potValue = analogRead(potPin);” is used to read the value of the potentiometer and stores it in the variable potValue
- “Serial.println(potValue);” prints potValue variable over the serial port

Python code:

- “import serial” imports the library needed to read data over the serial port
- “ser = serial.Serial('COM9', 9600)” is used to tell python to read data over the serial port and port com9 at a 9600 baud rate.
- “pot_value = ser.readline().decode().strip()” is used to read the potentiometer value sent over the serial port and store it in the variable pot_value
- “print("Potentiometer Value:", pot_value)” prints the potentiometer value to the display

Task 1

The experiment was carried out by:

1. Connecting the potentiometer and LED to the Arduino according to the required connections.
2. Writing and uploading an Arduino program to read the value of the potentiometer and print it over the serial port.
3. Using the serial plotter, the potentiometer value is plotted onto a graph in real time
4. Using Pycharm, a python script is written to read the value sent over the serial port and display the value.
5. The python script reads the potentiometer value. If the value exceeds 511, char ‘H’ will be sent over the serial port. If it’s below or equal to 511, char ‘L’ will be sent over serial port
6. The Arduino program receives the char sent over the serial port. If the char ‘H’ is received, the LED is turned on, if the char ‘L’ is received, the LED is turned off.

Arduino C++ code

- “int potValue = analogRead(potPin);” is used to read the value of the potentiometer and stores it in the variable potValue
- “pot_value = int(line)” converts the content of “line” variable into an integer and saves it to the variable “pot_value”
- “Serial.println(potValue);” prints potValue variable over the serial port
- “if (Serial.available() > 0)” checks if reading from serial port is available
- “char command = Serial.read();” read the char sent over the serial port and saves it to the variable “command”
- If and else if statement is used to turn LED on or off depending on the char received

Python code:

- “import serial” imports the library needed to read data over the serial port
- “ser = serial.Serial('COM9', 9600)” is used to tell python to read data over the serial port and port com9 at a 9600 baud rate.
- “line = ser.readline().decode().strip()” is used to read the potentiometer value sent over the serial port and store it in the variable line.
- “if line.isdigit():” checks if the received value is an integer
- “print("Potentiometer Value:", pot_value)” prints the potentiometer value to the display
- If and else statement is used to determine whether to send char ‘H’ or ‘L’ based on the potentiometer value

Experiment 2

The experiment was carried out by:

1. Connecting the servo motor to the Arduino according to the required connections.
2. Writing and uploading an Arduino program to read the value of the potentiometer and print it over the serial port.
3. Using the servo.h library, an Arduino program is written and uploaded to change the angle of the servo motor based on the angle value received over the serial port
4. Using pycharm, a python script is written to receive an angle value from the user and send it over the serial port to the Arduino program

Arduino C++ code

- “#include <Servo.h>” includes the library needed to move the servo
- “int angle = Serial.parseInt();” is used to read the value sent over the serial port and save it to an integer variable named “angle”
- “myServo.write(angle);” moves the servo motor to the angle saved in the “angle” variable

Python code:

- “import serial” imports the library needed to read data over the serial port
- “ser = serial.Serial('COM9', 9600)” is used to tell python to read data over the serial port and port com9 at a 9600 baud rate.
- “angle = input("Enter servo angle (0-180 deg, or 'q' to quit): ")” is used to receive an angle value from the user and save it to the variable “angle”
- “angle = int(angle)” converts the received angle to an integer
- “ ser.write(str(angle).encode())” sends the angle value over the serial port

Task 2

The experiment was carried out by:

1. Connecting servo motor and potentiometer to the Arduino according to the required connections.
2. Writing and uploading an Arduino program to read the value of the potentiometer and print it over the serial port. The Arduino program then maps the value of the potentiometer reading (0 to 1023) to the angle range of the servo motor (0 to 180) before changing the servo motor angle to match the mapped potentiometer value. The Arduino program also has a halting feature that turns off the LED and stops reading potentiometer if it receives the char ‘X’ over the serial port. If it receives the char ‘R’ it resumes operation
3. Using Pycharm, a python script is written to read the value sent over the serial port and display the value of the potentiometer. The python script includes a function to detect if the ‘x’ key is pressed which sends over the char ‘X’ over the serial port. If the ‘r’ key is pressed, the char ‘R’ is sent.
4. Connecting and LED to the Arduino according to the required connections
5. The python script was updated to send the char ‘H’ over the serial port if the potentiometer value is over 511 and send the char ‘L’ over the serial port if the potentiometer value is below or equal to 511.
7. The Arduino program is updated to receive the char sent over the serial port. If the char ‘H’ is received, the LED is turned on, if the char ‘L’ is received, the LED is turned off.
6. The python script is updated to use the matplotlib library to plot the potentiometer value on to a graph in real time

Arduino C++ code

- “#include <Servo.h>” includes the library needed to move the servo
- “bool isHalted = false;” creates a boolean variable set to halt. This is used to halt and resume the program when char ‘X’ or ‘R’ is received
- “if (Serial.available() > 0)” checks if reading from serial port is available
- “char command = Serial.read();” read the char sent over the serial port and saves it to the variable “command”

- If else statement is used to turn LED on or off, or halt the program according to the received char. If char ‘H’ is received LED is turned ON, if char ‘L’ is received LED is turned off, if char ‘X’ is received, boolean “isHalted” becomes true and if char ‘R’ is received boolean “isHalted” becomes false
- “if (isHalted)” checks if the boolean “isHalted” is true and if it is turns off the LED and ends the restarts the program loop stopping the potentiometer from being measured and the servo motor from being changed
- “int potValue = analogRead(potPin);” is used to read the value of the potentiometer and stores it in the variable potValue
- “Serial.println(potValue);” prints potValue variable over the serial port
- “int angle = map(potValue, 0,1023, 0,180)” is used to map the range of the potentiometer (0 to 1023) to the range of the servo motor (0 to 180)
- “myServo.write(angle);” moves the servo motor to the angle saved in the “angle” variable

Python code:

- “import serial” imports the library needed to read data over the serial port
- “import matplotlib.pyplot as plt” imports the matplotlib library to plot graphs as the class “plt”
- “ser = serial.Serial('COM9', 9600)” is used to tell python to read data over the serial port and port com9 at a 9600 baud rate.
- “plt.ion()” fuction is used to plot graphs in real time
- “def on_key(event):” fuction is called whenever a key is pressed
- “if event.key == 'x':” checks if the pressed key is ‘x’. If it is, then it sends the char ‘X’ is sent over the serial port
- “elif event.key == 'r':” checks if the pressed key is ‘r’. If it is then the char ‘R’ is sent over the serial port
- “fig.canvas.mpl_connect('key_press_event', on_key)” connects the key press event to the matplotlib window
- “line = ser.readline().decode().strip()” is used to read the potentiometer value sent over the serial port and store it in the variable “line”.
- “if line.isdigit():” checks if the received value is an integer
- “pot_value = int(line)” converts the content of “line” variable into an integer and saves it to the variable “pot_value”
- “print("Potentiometer Value:", pot_value)” prints the potentiometer value to the display
- “x_vals.append(len(x_vals))” plots the x-axis as the number of data given
- “y_vals.append(int(pot_value))” plots the y-axis as the value of the potentiometer stored in variable “pot_value”
- “ax.clear()” clears the graph
- “ax.plot(x_vals, y_vals, marker='o')” plots the graph based on the previously given values

- “ax.set_ylim(0, 1023)” sets the limit of the y-axis from 0 to 1023 which is the range of potentiometer readings
- “plt.pause(0.1)” updates the plot every 0.1 second
- If else statement is used to determine whether to send char ‘H’ or ‘L’ based on the potentiometer value

DATA COLLECTION & ANALYSIS

Experimental Data Table

During the real-time serial transmission test (task 2.3), analog readings from the potentiometer (connected to analog pin `A0`) were successfully captured by Python via PyCharm and plotted using `matplotlib`. The table below samples representative operating points from the live graphical capture, showing the corresponding ADC values, calculated servo angles, and automated LED states:

Sample Point / Phase	Raw ADC Reading (0−1023)	Calculated Voltage (V)	Servo Motor Angle (0°−180°)	LED State (>511 Threshold)
Initial Resting State	280	1.37 V	49°	OFF (`L`)
Midpoint Threshold	511	2.50 V	90°	OFF (`L`) / Transitioning
Upper Linear Sweep	700	3.42 V	123°	ON (`H`)
Maximum Rotation	1000	4.88 V	176°	ON (`H`)
Minimum Rotation (Drop)	0	0.00 V	0°	OFF (`L`)

Technical Analysis & Mathematical Formulations

- **Analog-to-Digital Conversion (ADC):** The Arduino Uno utilizes a 10-bit ADC, mapping an input voltage range of 0V to 5V into discrete integer values from 0 to 1023:

  $$Vin = ADC\ Value/1023\ X\ 5.0V$$

- **Servo Angle Mapping:** To convert the raw potentiometer reading into a physical angular displacement suitable for the servomotor 0° to 180°, the Arduino script executes a linear scaling function:

  $$Angle = map(potValue, 0, 1023, 0, 180) = (potValue/1023)\ X\ 180°$$

- **Threshold Derivation:** The conditional trigger for the digital LED output is defined at precisely half of the ADC's maximum operational range:

  $$Threshold = 1023/2 = 511.5 = approx\ 511$$

When potValue > 511, Python transmits byte `'H'`, driving digital pin `HIGH`; otherwise, byte `'L'` is transmitted, driving pin `LOW`.

RESULTS & DISCUSSION

Hardware Behavior and Evaluation Against Hypothesis

- **Real-Time Graphical Visualization:** The generated `matplotlib` interactive plot (captured in the experimental run) successfully displayed a continuous real-time waveform of the potentiometer turning cycles. As the knob was rotated manually from a low position (~280 ADC) up to its maximum limit (~1000 ADC) and back down to zero, the data points populated smoothly on the live graph with minimal latency.
- **Synchronized Electro-Mechanical Response:** The hardware behaved exactly as hypothesized. As the potentiometer value crossed the midpoint threshold (511), the LED switched states instantaneously based on the serial command loop, while the servo motor concurrently tracked the rotation angle smoothly across its 0° to 180° span.

Error Sources and Solutions

During the integration and testing phase, several practical engineering challenges were encountered and resolved:

- **Matplotlib Rendering in PyCharm:**
  - *Issue:* Initially, the live plot failed to display or update dynamically when running the script inside PyCharm.
  - *Root Cause:* PyCharm's default integrated tool window backend intercepts GUI pop-ups or locks the event loop needed for interactive real-time updating (`plt.ion()`).
  - *Solution:* Disabled the "Show plots in tool window" setting in PyCharm's Python scientific configurations, forcing `matplotlib` to render via a separate, native interactive OS window backend.
- **Signal Jitter Near the Threshold:**
  - *Issue:* Minor electrical noise or resistance fluctuations right around the 511 threshold caused rapid toggling (chattering) of the LED state.
  - *Solution:* Adding a small debounce delay (`delay(100)` in Arduino loop and `plt.pause(0.1)` in Python) stabilized the serial transmission frequency and filtered out high-frequency transient noise.
- **Serial Communication & Keyboard Interception:**
  - *Issue:* Capturing external keyboard commands simultaneously with continuous serial reading proved complex within a standard blocking loop structure.
  - *Solution:* Delegated threshold logic and termination control directly through structured serial data handshakes (`KeyboardInterrupt` handling) to ensure connection integrity and safe port closure (`ser.close()`).

CONCLUSION

In this experiment, serial communication between an Arduino Uno and a Python environment was successfully established to control and monitor mechatronic actuators and sensors.

- **Experiment 1 & Task 1** demonstrated the ability to read analog sensor inputs from a potentiometer on the Arduino and stream them in real-time to a computer via USB serial, using the `pyserial` library for Python data acquisition.
- **Experiment 2** successfully implemented bidirectional or computer-driven serial control, allowing user inputs from the Python terminal to dictate the target angular position of a servo motor connected to a PWM pin (Pin 9).
- **Task 2** further extended these concepts by integrating real-time graphical visualization using Python's `matplotlib` library, enabling continuous plotting of sensor data alongside threshold-based feedback controls (such as LED indicators).

Overall, the experiment verified the foundational principles of hardware-software integration, showcasing how microcontrollers and high-level software languages like Python can interface effectively for embedded system monitoring and control applications.

RECOMMENDATIONS

To improve future iterations of this experiment and optimize system performance, the following recommendations are proposed:

1. **Implement Proper Baud Rate and Buffer Management:** Ensure that both the Arduino code and the Python script strictly share the same baud rate (e.g., 9600) to prevent data corruption or frame errors during serial transmission. Adding a short delay (like `delay(15)` or `delay(50)`) on the Arduino side can prevent buffer overflows when transmitting high-frequency data streams.
2. **Avoid Port Conflict Overlaps:** Always ensure that the Arduino IDE's **Serial Monitor** or **Serial Plotter** is completely closed before running the Python script. Operating both simultaneously will result in a `Permission Error` (Port busy) because the COM port can only be handled by a single application at a time.
3. **Incorporate Hardware Debouncing and Filtering:** Potentiometers are prone to electrical noise, which can cause jittery readings in the serial monitor or erratic servo movements. Implementing a simple moving average filter in Python or a hardware capacitor filter can stabilize the analog readings.
4. **Enhance Error Handling in Python:** Add robust exception handling blocks (such as `try-except-ValueError` wrappers) in the Python script to gracefully manage unexpected non-integer inputs or sudden disconnections of the USB serial cable without crashing the application.

REFERENCES

- Prahbu, S. (2026). How Servo Motor Works & Interface It With Arduino https://lastminuteengineers.com/servo-motor-arduino-tutorial/
- Arduino. (2026). Tutorials - From beginner to Advanced https://docs.arduino.cc/tutorials/
- Cornelam. (2015). Arduino Servo Motors https://www.instructables.com/Arduino-Servo-Motors/

APPENDICES

Video links:
https://drive.google.com/drive/folders/1Nyr55JrTSgEieY_zFmxJ4xZPBjsIsvQ3?hl=ID

ACKNOWLEDGEMENT

We would like to express our deepest appreciation to our instructor and lab personnel for their guidance and support throughout this experiment.

STUDENT DECLARATION

Certificate of Originality and Authenticity

This is to certify that we are responsible for the work submitted in this report, that the original work is our own except as specified in the references and acknowledgments, and that the original work contained herein has not been undertaken or done by unauthorized sources or persons.

SIGNATURE	STUDENT DETAILS	CHECKLIST
	Name: Harith Naufal Bin Ahmad Shukri Matric No: 2413381	Read [/] Understand [/] Agree [/]
	Name: Iman Bin Salman Matric No: 2418123	Read [/] Understand [/] Agree [/]
	Name: Luqman Hakimi Bin Suhaizi Matric No: 2411349	Read [/] Understand [/] Agree [/]
	Name: Muhamad Haziq Bin Mohd Shahril Matric No: 2418929	Read [/] Understand [/] Agree [/]